Data Structures Reviewer:
Object-Oriented Programming (OOP):
The four pillars of OOP are abstraction, encapsulation, inheritance, and polymorphism.

Abstraction is the process of showing only the essential features of an entity and hiding irrelevant information. It reduces complexity, avoids code duplication, eases maintenance, and increases security.

Encapsulation is the process of wrapping data and member functions together into a single class.

Inheritance is the ability to create a new class from an existing one, where the new class acquires the properties of the original.

Polymorphism is the concept of having many forms.

Arrays:

An array is a way to store multiple values of the same type in a single variable.

Array elements are accessed using their index, which starts at 0.

Java uses pass-by-value for parameter passing. For primitive types, a copy of the value is passed, so changes to the parameter do not affect the original value. For reference types (like objects and arrays), a copy of the reference (memory address) is passed, so while you cannot change the reference to a different object, you can modify the object itself.

Linked Lists
A Linked List is a linear data structure composed of a series of connected nodes.

A node is the basic unit of a linked list and has two parts: data and a next pointer. The data holds the value, and the next pointer stores the memory address of the next node.

Advantages over arrays: Linked lists are not fixed in size, as memory is allocated dynamically. They also allow for efficient insertion and deletion in constant time.

Disadvantages over arrays: Linked lists only allow for sequential access, meaning you have to go through the links to find a specific element. Additionally, each node requires extra memory to store the next pointer.

The three most common types of linked lists are

Singly Linked List, Doubly Linked List, and Circular Linked List.

Singly Linked List
A singly linked list is a unidirectional linked list, meaning you can only traverse it in one direction, from the head to the tail. Each node contains two parts: the data and a pointer that references the next node in the sequence. The last node in the list points to a NULL value to signify the end of the list.

Doubly Linked List
A doubly linked list allows for traversal in both directions—forward and backward. Each node in this type of list contains three parts: the data, a pointer to the next node, and a pointer to the previous node. The previous pointer of the first node and the next pointer of the last node are both NULL.

Circular Linked List
A circular linked list is a variation where the last node points back to the first node, creating a continuous loop. This means there is no NULL pointer to mark the end of the list. This type can be either singly or doubly linked, depending on whether it has one or two pointers per node.

Sorting Algorithms:

Sorting is the process of arranging elements in a specific order, such as numerical or alphabetical.

Sorting algorithms are important because they reduce the complexity of problems and are used in searching algorithms, databases, and more.

Sorting algorithms can be classified in the following ways:

Comparison-based: These algorithms sort by comparing elements. Examples include Bubble Sort, Insertion Sort, Selection Sort, Quick Sort, Merge Sort, and Heap Sort.

Non-comparison-based: These algorithms sort without directly comparing elements. Examples are Counting Sort, Radix Sort, and Bucket Sort.

Stable vs. Unstable: Stable sorting algorithms maintain the relative order of equal elements (e.g., Bubble Sort, Merge Sort, Insertion Sort). Unstable algorithms do not (e.g., Quick Sort, Heap Sort, Selection Sort).

Recursive vs. Iterative: Recursive algorithms use recursive calls (e.g., Merge Sort, Quick Sort). Iterative algorithms use loops (e.g., Bubble Sort, Selection Sort).